

GGL90 Python Port Reference: Code Flow and Fortran-to-Python Mapping

1D Mixing Model Development Team

August 2026

Contents

1	Introduction	2
1.1	Purpose of This Document	2
1.2	Scope of the Port	2
1.3	How to Use This Document	3
2	Overview of Code Flow	3
2.1	Algorithm Flowchart	3
2.2	Main Entry Points	3
2.3	Calling Sequence	4
3	Step-by-Step Code Mapping	4
3.1	Parameter Initialization	4
3.2	Stratification and Shear Calculation	5
3.3	Mixing Length Calculation	6
3.4	TKE Evolution (Prognostic Step)	7
3.5	Eddy Diffusivity Calculation	8
3.6	Boundary Conditions	9
3.6.1	Surface Boundary Conditions	9
3.6.2	Bottom Boundary Conditions	9
3.7	Static Instability Handling	10
4	Supporting Infrastructure	11
4.1	Parameter Handling	11
4.2	Grid and Coordinate System	11
4.3	Physical Constants	12
4.4	Boundary Condition Details	12
5	File-to-File Correspondence Table	12
6	Quick Reference: Key Variables	12
6.1	Variable Name Mappings	12
6.2	Unit Conversions	12
6.3	Sign Convention Differences	13

1 Introduction

1.1 Purpose of This Document

This document serves as a **reference guide** for mapping the MITgcm GGL90 turbulence closure scheme (Fortran) to its Python port in the 1D Column Model. The document is organized by algorithm flow, providing file and line number references for each major computational step. Use this document to:

- Trace where specific GGL90 capabilities from MITgcm are implemented in Python
- Understand the computational flow of the GGL90 mixing scheme
- Locate corresponding code sections when comparing Fortran and Python implementations
- Identify file-to-file correspondence between the two implementations

1.2 Scope of the Port

The Python port implements the GGL90 turbulence closure scheme for a 1D ocean column. The port faithfully reproduces the core physics and numerical methods from MITgcm's `pkg/gg190`, adapted for the 1D vertical column context.

Included in the port:

- TKE prognostic evolution with implicit time stepping
- Mixing length computation (multiple limiting methods)
- Eddy viscosity and diffusivity calculation
- TKE budget terms (production, buoyancy, dissipation)
- Surface and bottom boundary conditions for TKE
- Static instability handling (convective adjustment)
- Stratification (N^2) and shear (S^2) diagnostics

Excluded from the 1D port:

- Horizontal mixing and advection
- IDEMIX internal wave energy model
- Langmuir circulation parameterization
- Grid-scale noise smoothing
- 3D-specific features (requires horizontal neighbors)

1.3 How to Use This Document

1. **Section 2 (Overview):** Start here to understand the high-level algorithm flow and main entry points
2. **Section 3 (Step-by-Step Mapping):** Use this to find where each major computational step lives in both codebases
3. **Section 4 (Infrastructure):** Consult for parameter handling, grid conventions, and boundary conditions
4. **Section 5 (File Correspondence):** Quick lookup table for file-to-file mapping
5. **Section 6 (Variable Reference):** Look up variable name mappings and sign conventions

2 Overview of Code Flow

2.1 Algorithm Flowchart

The GGL90 scheme computes vertical mixing coefficients through the following sequence:

1. **Parameter Initialization:** Load scheme parameters from configuration files
2. **Stratification and Shear Calculation:** Compute N^2 (buoyancy frequency squared) and S^2 (vertical shear squared)
3. **Mixing Length Calculation:** Compute turbulent mixing length ℓ with physical limits
4. **TKE Evolution:** Step turbulent kinetic energy forward in time (prognostic)
5. **Eddy Diffusivity Calculation:** Compute κ_m (viscosity) and κ_h (diffusivity)
6. **Boundary Conditions:** Apply surface and bottom boundary conditions
7. **Static Instability Handling:** Apply convective adjustment when needed

2.2 Main Entry Points

Python:

- Module: `GGL90_ML/GGL90_PY/ggl90_core_driver.py`
- Class: `GGL90Driver` (lines 81-356)
- Method: `compute_mixing()` (lines 230-356)
- Called from: `main/mixing_adapter.py` via unified interface

Fortran:

- File: `pkg/ggl90/ggl90_calc.F`
- Subroutine: `GGL90_CALC` (lines 13-1177)
- Called from: `model/src/do_oceanic_phys.F`

2.3 Calling Sequence

Python calling sequence:

```
unified_driver.py::step_forward()
-> mixing_adapter.py::compute_mixing_coefficients()
   -> ggl90_core_driver.py::GGL90Driver.compute_mixing()
       -> physics_basis.py::compute_buoyancy_frequency_squared()
       -> ggl90_scheme_specific.py::GGL90MixingLength.compute()
       -> ggl90_mixing_coefficients.py::compute_viscosity_diffusivity()
       -> ggl90_core_driver.py::step_tke_forward()
```

Fortran calling sequence:

```
do_oceanic_phys.F::DO_OCEANIC_PHYS()
-> ggl90_calc.F::GGL90_CALC()
   -> ggl90_mixinglength.F::GGL90_MIXINGLENGTH()
```

3 Step-by-Step Code Mapping

This section provides detailed line-by-line mapping for each major computational step in the GGL90 algorithm.

3.1 Parameter Initialization

Purpose: Load and initialize GGL90 scheme parameters, including closure coefficients (c_k , c_ε , α), minimum values ($\text{TKE}_{\min}$, $\ell_{\min}$), and numerical control flags.

Python Implementation:

- File: GGL90_ML/GGL90_PY/ggl90_parameters.py
- Class: GGL90Parameters (lines 1-300+)
- Configuration files: configuration_yamls/ggl90_default.yaml, ggl90_eccov4r4.yaml
- Key parameters loaded: ck, ceps, alpha, tke_min, mixing_length_min, mxl_max_flag, mxl_surf_flag

Fortran Source:

- File: pkg/ggl90/ggl90_readparms.F (lines 1-500+)
- Subroutine: GGL90_READPARMS reads namelist GGL90_PARM01
- File: pkg/ggl90/ggl90_init_fixed.F (lines 1-50+)
- Subroutine: GGL90_INIT_FIXED initializes fixed parameters
- Header: pkg/ggl90/GGL90.h defines parameter variables

Key Parameters:

- GGL90ck (Fortran) $\leftrightarrow$ ck (Python): Turbulent velocity scale coefficient (default: 0.1)

- GGL90ceps (Fortran) $\leftrightarrow$ ceps (Python): Dissipation coefficient (default: 0.7)
- GGL90alpha (Fortran) $\leftrightarrow$ alpha (Python): TKE diffusivity coefficient (default: 1.0)
- GGL90TKEmin (Fortran) $\leftrightarrow$ tke_min (Python): Minimum TKE (default: $1e-11 \text{ m}^2/\text{s}^2$)
- GGL90mixingLengthMin (Fortran) $\leftrightarrow$ mixing_length_min (Python): Minimum mixing length (default: $1e-8 \text{ m}$)

3.2 Stratification and Shear Calculation

Purpose: Compute the buoyancy frequency squared (N^2) and vertical shear squared (S^2), which drive mixing suppression and production respectively.

Python Implementation:

- File: main/eos.py
- Function: compute_ggl90_buoyancy_frequency_squared() (lines 200-300+)
- Called from: ggl90_core_driver.py::compute_mixing() (line 311)
- Key computation: Uses **potential density gradients** (not in-situ density)
- File: main/physics_basis.py
- Function: compute_vertical_shear_squared() (lines 50-100+)
- Called from: ggl90_core_driver.py::compute_mixing() (line 314)

Fortran Source:

- File: pkg/ggl90/ggl90_calc.F
- Subroutine: GGL90_CALC
- N^2 computation: lines 340-360 (uses `sigmaR` = potential density gradient)
- Vertical shear computation: lines 476-495
- Key variables:
 - `Nsquare(i,j,k)` $\leftarrow$ line 347: `gravity*gravitySign*recip_rhoConst * sigmaR`
 - `verticalShear(i,j)` $\leftarrow$ lines 476-495: $(du/dz)^2 + (dv/dz)^2$

Implementation Notes:

- Both implementations use potential density gradients (MITgcm's `sigmaR`), not in-situ density
- This is critical for avoiding spurious mixing from compressibility effects
- Python port was corrected in July 2026 to match this convention

3.3 Mixing Length Calculation

Purpose: Compute the turbulent mixing length scale ℓ based on TKE, stratification, and distance from boundaries. The mixing length determines the spatial scale of turbulent eddies.

Python Implementation:

- File: `GGL90_ML/GGL90_PY/ggl90_scheme_specific.py`
- Class: `GGL90MixingLength` (lines 21-231)
- Method: `compute()` (lines 37-101)
- Called from: `ggl90_core_driver.py::compute_mixing()` (lines 323-325)
- Limiting methods:
 - Method 0: `_limit_method_0()` (lines 103-123) — water column depth limit
 - Method 1: `_limit_method_1()` (lines 125-145) — minimum distance to surface/bottom
 - Method 2: `_limit_method_2()` (lines 147-194) — two-way sweep (Blanke & Delecluse 1993)

Fortran Source:

- File: `pkg/ggl90/ggl90_mixinglength.F`
- Subroutine: `GGL90_MIXINGLENGTH` (lines 10-380+)
- Called from: `ggl90_calc.F::GGL90_CALC` (line 520+)
- Initial length computation: lines 100-150+ (equation 2.35 from Gaspar et al. 1990)
- Limiting logic: controlled by `mx1MaxFlag` parameter
 - Method 0: lines 200-250+ (simple depth limit)
 - Method 1: lines 250-300+ (distance to boundaries)
 - Method 2: lines 300-380+ (two-way sweep algorithm)

Key Computation:

Initial mixing length from TKE and stratification (Gaspar et al. 1990, Eq. 2.35):

$$\ell = \sqrt{2} \frac{\sqrt{\text{TKE}}}{\sqrt{N^2}} \quad (1)$$

Limiting ensures physical constraints:

- Method 0: $\ell \leq H$ (total water column depth)
- Method 1: $\ell \leq \min(z_{\text{surf}}, z_{\text{bot}})$ (distance to nearest boundary)
- Method 2: Two-way sweep for smooth vertical variation

3.4 TKE Evolution (Prognostic Step)

Purpose: Advance turbulent kinetic energy forward in time using an implicit tridiagonal solver. This is the core prognostic equation of GGL90, distinguishing it from diagnostic schemes like KPP.

Python Implementation:

- File: `GGL90_ML/GGL90_PY/ggl90_core_driver.py`
- Method: `GGL90Driver.step_tke_forward()` (lines 104-228)
- Called from: `compute_mixing()` (lines 339-342)
- Tridiagonal solver: `main/shared_column_solver.py::solve_tridiagonal()`
- System assembly: lines 166-218 (coefficients a , b , c , rhs)
- Surface boundary condition: lines 205-210 (Dirichlet: $\text{TKE} = m^2 u_*^2$)
- Bottom boundary condition: lines 214-218 (Dirichlet or Neumann, controlled by parameter)

Fortran Source:

- File: `pkg/ggl90/ggl90_calc.F`
- Subroutine: `GGL90_CALC`
- TKE equation assembly: lines 580-700+ (implicit tridiagonal system)
- Dissipation term: line 600+ (`TKEdissipation = ceps * TKE^(3/2) / L`)
- Production term: line 607+ (`KappaM * verticalShear`)
- Buoyancy term: line 615+ (`-KappaH * Nsquare`)
- Tridiagonal solve: lines 750-850+ (Thomas algorithm or equivalent)
- Surface BC: lines 630-650+ (Dirichlet condition at surface)
- Bottom BC: lines 720-740+ (controlled by `GGL90_dirichlet`)

Key Equation:

The TKE prognostic equation:

$$\frac{\partial \text{TKE}}{\partial t} = P + B - \varepsilon + \frac{\partial}{\partial z} \left(\kappa_E \frac{\partial \text{TKE}}{\partial z} \right) \quad (2)$$

where:

- $P = \kappa_m S^2$ (shear production)
- $B = -\kappa_h N^2$ (buoyancy term, positive for unstable stratification)
- $\varepsilon = c_\varepsilon \text{TKE}^{3/2} / \ell$ (dissipation, treated implicitly)
- $\kappa_E = \alpha \kappa_m$ (TKE diffusivity)

Implementation Notes:

- Dissipation is split into implicit (`impl_diss_fac`) and explicit (`expl_diss_fac`) parts for numerical stability
- Surface TKE boundary condition: $\text{TKE}_{\text{surf}} = m^2 u_*^2$ where $m^2 \approx 3.75$ (parameter `m2`)
- The tridiagonal system is solved using a shared Thomas algorithm solver (single source of truth)

3.5 Eddy Diffusivity Calculation

Purpose: Compute eddy viscosity (κ_m) and eddy diffusivity (κ_h) from TKE and mixing length. These coefficients drive momentum and tracer mixing.

Python Implementation:

- File: `GGL90_ML/GGL90_PY/ggl90_mixing_coefficients.py`
- Function: `compute_viscosity_diffusivity()` (lines 17-111)
- Called from: `ggl90_core_driver.py::compute_mixing()` (lines 328-331)
- Main loop: lines 96-109 (compute coefficients for each vertical level)
- Turbulent Prandtl number: lines 82-92 (function of Richardson number)
- Background floor: applied at lines 104-109 (matches MITgcm behavior)

Fortran Source:

- File: `pkg/ggl90/ggl90_calc.F`
- Subroutine: `GGL90_CALC`
- Viscosity computation: lines 441-461
- Key line 448: `KappaM(i,j) = GGL90ck*GGL90mixingLength(i,j,k)*SQRTTKE(i,j,k)`
- Prandtl number: lines 560-580 (function of local Richardson number)
- Diffusivity from viscosity: `KappaH = KappaM / TKEPrandtlNumber`
- Background floor: line 461+ (`MAX(KappaM, viscArNr(k))`)

Key Computations:

Eddy viscosity:

$$\kappa_m = \max \left(c_k \ell \sqrt{\text{TKE}}, \nu_{\text{background}} \right) \quad (3)$$

Eddy diffusivity:

$$\kappa_h = \max \left(\min \left(\frac{\kappa_m}{\text{Pr}_T}, \kappa_{\text{max}} \right), \kappa_{\text{background}} \right) \quad (4)$$

Turbulent Prandtl number (for stable stratification, $\text{Ri} \geq 0.2$):

$$\text{Pr}_T = \min(10, 5\text{Ri}) \quad (5)$$

where Richardson number:

$$\text{Ri} = \frac{N^2}{S^2 + \epsilon} \quad (6)$$

Implementation Notes:

- Background viscosity/diffusivity (MITgcm’s `viscArNr/diffKrNrS`) are applied as floor values
- Maximum caps (`visc_max, diff_max`) prevent unrealistic values
- Index 0 (surface face) is set to 0 (no surface diffusive flux)
- Interior faces $k=1..nz-1$ carry the mixing coefficients

3.6 Boundary Conditions

Purpose: Apply physical boundary conditions at the ocean surface and bottom for TKE evolution and mixing coefficients.

3.6.1 Surface Boundary Conditions

Python Implementation:

- File: `GGL90_ML/GGL90_PY/ggl90_core_driver.py`
- Surface TKE: `step_tke_forward()` lines 205-210
- Formula: `surf_tke = max(m2 * u_star_sq, tke_surf_min)`
- Parameter $m2 \approx 3.75$ (default)
- Parameter `tke_surf_min` $\approx 1e-12 \text{ m}^2/\text{s}^2$ (prevents zero TKE)

Fortran Source:

- File: `pkg/ggl90/ggl90_calc.F`
- Surface TKE: lines 630-650+
- Uses same formula with `GGL90m2` parameter
- Applied as Dirichlet boundary condition in tridiagonal system

3.6.2 Bottom Boundary Conditions

Python Implementation:

- File: `GGL90_ML/GGL90_PY/ggl90_core_driver.py`
- Bottom TKE: `step_tke_forward()` lines 214-218
- Two options controlled by `use_dirichlet` parameter:
 - Dirichlet: $\text{TKE}_{\text{bottom}} = \text{tke_bottom}$ (typically $1e-12 \text{ m}^2/\text{s}^2$)
 - Neumann: $\partial \text{TKE} / \partial z = 0$ (zero flux)

Fortran Source:

- File: `pkg/ggl90/ggl90_calc.F`
- Bottom TKE: lines 720-740+

- Controlled by `GGL90_dirichlet` logical flag
- Same Dirichlet vs. Neumann choice

Implementation Notes:

- Surface friction velocity u_* is computed from wind stress: $u_*^2 = \sqrt{\tau_x^2 + \tau_y^2} / \rho_0$
- Surface TKE boundary condition represents TKE generation by surface wave breaking and wind stirring
- Bottom boundary condition can be Dirichlet (fixed small TKE near bottom) or Neumann (isolating)
- Default MITgcm configuration uses Dirichlet bottom BC

3.7 Static Instability Handling

Purpose: Handle convective instability ($N^2 \leq 0$) through enhanced mixing, representing unresolved convective plumes.

Python Implementation:

- File: `main/convective_adjustment.py`
- Function: `apply_convective_adjustment()` (lines 1-100+)
- Called from: `unified_driver.py` after mixing coefficient computation
- Parameter: `ivdc_kappa` controls enhanced diffusivity for unstable regions
- When $N^2 \leq 0$, diffusivity is augmented: $\kappa_{\text{total}} = \kappa_{\text{GGL90}} + \kappa_{\text{ivdc}}$

Fortran Source:

- File: `pkg/ggl90/ggl90_calc.F`
- Integrated into main calculation: lines 400-430+ (approximate)
- Uses `ivdc_kappa` parameter (typically $0.1 \text{ m}^2/\text{s}$ for convective regions)
- Applied when `Nsquare < 0`
- Can also use implicit vertical diffusion solver for convective adjustment

Implementation Notes:

- Static instability occurs when dense water overlies light water ($N^2 \leq 0$)
- In reality, this triggers rapid overturning on sub-grid scales
- The parameterization represents this with very large vertical diffusivity
- Python port uses separate convective adjustment module for clarity
- Both implementations add `ivdc_kappa` to the turbulent diffusivity when $N^2 \leq 0$

4 Supporting Infrastructure

4.1 Parameter Handling

Python Approach:

- YAML-based configuration files in `configuration_yamls/`
- `ggl90_default.yaml`: MITgcm default parameters
- `ggl90_eccov4r4.yaml`: ECCO v4 Release 4 tuned parameters
- Parameters loaded into `GGL90Parameters` dataclass
- All parameters have documented units and default values

Fortran Approach:

- Namelist-based configuration (`data.ggl90` file)
- Namelist group: `GGL90_PARM01`
- Parameters read in `ggl90_readparms.F`
- Defaults defined in `GGL90.h` header file
- Parameters stored in COMMON blocks

4.2 Grid and Coordinate System

Coordinate Conventions (both implementations):

- **z-axis**: Positive upward, $z = 0$ at surface
- **depth**: Positive downward ($\text{depth} = -z$)
- **Vertical index k**: $k=0$ is shallowest (surface), $k=nz-1$ is deepest (bottom)
- **Vertical spacing**: $\text{dz}[k]$ is thickness of cell k

Staggering Conventions:

- Tracers (T, S) at cell centers
- Velocities (u, v) at cell centers (1D column)
- TKE at cell interfaces (W-points in MITgcm terminology)
- Mixing coefficients (κ_m, κ_h) at cell interfaces
- N^2, S^2 computed at cell interfaces
- Mixing length ℓ at cell interfaces

Python vertical arrays:

- `z[k]`: z-coordinate of cell center k [m]

- `depth[k]`: depth of cell center k [m], positive
- `dz[k]`: thickness of cell k [m]
- `tke[k]`: TKE at interface k (between cells $k-1$ and k) [m^2/s^2]
- `kappa_m[k]`, `kappa_h[k]`: mixing coefficients at interface k [m^2/s]

Fortran vertical arrays:

- `rC(k)`: z -coordinate of cell center k [m]
- `rF(k)`: z -coordinate of cell interface k [m]
- `drF(k)`: thickness of cell k [m]
- `GGL90TKE(i,j,k)`: TKE at interface k [m^2/s^2]
- `GGL90viscAz(i,j,k)`, `GGL90diffKr(i,j,k)`: mixing coefficients at interface k [m^2/s]

4.3 Physical Constants

Common constants (both implementations):

- `gravity` (Python) = `gravity` (Fortran) = 9.81 m/s^2 (or 9.80665 m/s^2)
- `rho_const` (Python) = `rhoConst` (Fortran) = 1029.0 kg/m^3 (reference density)
- `sqrt_two` (Python) = `SQRT2` (Fortran) = $\sqrt{2} \approx 1.414$

GGL90-specific constants:

- `gg190_eps` (Python) = `GGL90eps` (Fortran) = $1\text{e-}20$ (numerical regularization)

4.4 Boundary Condition Details

See Section 3.6 for detailed boundary condition implementation. Summary:

- **Surface:** Dirichlet for TKE ($\text{TKE} = m^2 u_*^2$), zero flux for mixing coefficients
- **Bottom:** Dirichlet or Neumann for TKE (parameter-controlled), zero flux for mixing coefficients
- **Lateral:** Not applicable in 1D column model

5 File-to-File Correspondence Table

6 Quick Reference: Key Variables

6.1 Variable Name Mappings

6.2 Unit Conversions

No unit conversions are needed between the Python and Fortran implementations. Both use SI units consistently:

Python Module	MITgcm Fortran File(s)
ggl90_core_driver.py	pkg/ggl90/ggl90_calc.F (main orchestrator)
ggl90_scheme_specific.py	pkg/ggl90/ggl90_mixinglength.F, portions of ggl90_calc.F (TKE terms)
ggl90_mixing_coefficients.py	pkg/ggl90/ggl90_calc.F (lines 441-580+, viscosity/diffusivity)
ggl90_parameters.py	pkg/ggl90/ggl90_readparms.F, pkg/ggl90/GGL90.h (parameter definitions)
main/eos.py	model/src/find_rho.F (equation of state)
main/physics_basis.py	Portions of ggl90_calc.F (N^2 , S^2 diagnostics)
main/shared_column_solver.py	model/src/solve_tridiagonal.F (or equivalent)
main/convective_adjustment.py	model/src/convective_adjustment.F, ivdc_kappa logic in ggl90_calc.F
Fortran files NOT ported (3D/horizontal features):	
—	pkg/ggl90/ggl90_idemix.F (internal wave model)
—	pkg/ggl90/ggl90_add_stokesdrift.F (Langmuir circulation)
—	pkg/ggl90/ggl90_exchanges.F (horizontal exchanges)

Table 1: Python-to-Fortran file correspondence for GGL90 port

- Length: meters [m]
- Time: seconds [s]
- Temperature: degrees Celsius [°C]
- Salinity: practical salinity units [psu]
- Velocity: meters per second [m/s]
- TKE: square meters per square second [m^2/s^2]
- Mixing coefficients: square meters per second [m^2/s]

6.3 Sign Convention Differences

No **sign convention differences** between Python and Fortran implementations. Both use:

- z-axis positive upward ($z = 0$ at surface, $z < 0$ below surface)
- depth positive downward ($\text{depth} = -z$)

Python Name	Fortran Name	Description
tke	GGL90TKE	Turbulent kinetic energy [m^2/s^2]
kappa_m	GGL90viscAz	Eddy viscosity [m^2/s]
kappa_h	GGL90diffKr	Eddy diffusivity [m^2/s]
mixing_length	GGL90mixingLength	Mixing length [m]
n_square	Nsquare	Buoyancy frequency squared [s^{-2}]
shear_square	verticalShear	Vertical shear squared [s^{-2}]
production	TKEproduction (diagnostic)	TKE shear production [m^2/s^3]
buoyancy	TKEbuoyancy (diagnostic)	TKE buoyancy term [m^2/s^3]
dissipation	TKEdissipation	TKE dissipation [m^2/s^3]
u_star_sq	uStarSquare	Friction velocity squared [m^2/s^2]
kappa_e	KappaE	TKE diffusivity [m^2/s]
z	rC (cell centers)	Vertical coordinate [m]
depth	-rC	Depth (positive downward) [m]
dz	drF	Cell thickness [m]
Parameters:		
ck	GGL90ck	Turbulent velocity scale coeff. (default: 0.1)
ceps	GGL90ceps	Dissipation coefficient (default: 0.7)
alpha	GGL90alpha	TKE diffusivity coefficient (default: 1.0)
tke_min	GGL90TKEmin	Minimum TKE [m^2/s^2] (default: $1\text{e-}11$)
mixing_length_min	GGL90mixingLengthMin	Minimum ℓ [m] (default: $1\text{e-}8$)
m2	GGL90m2	Surface TKE coefficient (default: 3.75)
mxl_max_flag	mxlMaxFlag	Mixing length limiting method (0, 1, 2, 3)
mxl_surf_flag	mxlSurfFlag	Force surface mixing length flag

Table 2: Python-to-Fortran variable name mappings

- Vertical gradients computed consistently
- Buoyancy term $B = -\kappa_h N^2$ (negative for stable stratification)

Critical fix (July 2026): The Python port initially used in-situ density gradients for N^2 , which introduced spurious compressibility effects. The port was corrected to use potential density gradients (MITgcm’s `sigmaR`), matching the Fortran implementation exactly.